Small Business Inventory and Profitability Simulator

Python group project | 4-6 members | Medium difficulty

Complete all core requirements and exactly one of the three alternative extensions. Submit one application per group. All members are responsible for understanding the complete codebase.

1. Overview

Build a command-line application for a fictional shop with a small product catalogue. Analyse historical sales, simulate future demand and replenishment, and compare inventory policies. The central question is how to balance the cost of keeping stock against losing sales when products are unavailable.

Complete the compulsory core and choose exactly one extension. The project covers fundamental data types, functions, classes with inheritance, exceptions, NumPy and basic SQL. Architecture, interfaces and report layout are open design choices.

Economic concepts

Revenue: price multiplied by units sold. **Cost of goods sold:** purchase cost multiplied by units sold. **Gross profit:** revenue minus cost of goods sold. **Holding cost:** a modelled cost of keeping unsold stock. **Lost sales:** demand that available stock cannot satisfy.

Profit and cash flow differ. This project computes operating results under a simplified model, not full business accounting. Unsold purchased units are inventory, not cost of goods sold.

2. Student specifications: configuration

Read a TOML file containing input/output/database paths, simulation length, random seed, demand-generation settings, holding cost per unit per day, fixed cost per replenishment order, inventory-policy parameters, and the selected extension with its parameters.

Validate keys, types and values; resolve relative paths against the configuration file directory. Prices and purchase costs remain constant within a simulation, except for an extension that explicitly changes selling prices.

Core workflow

Import and validate CSV files, store them in SQLite, summarise historical sales with SQL, generate demand and compare policies using NumPy, then export CSV results and a readable text report. Provide documented command-line commands or options. No manual database editing or graphical interface is required.

Student specifications: input and SQL

Product catalogue

```
product_id,name,category,purchase_cost,selling_price,initial_stock
P001,Notebook,Stationery,1.20,2.50,40
P002,Pen,Stationery,0.30,0.80,100
```

Historical daily sales

```
date,product_id,units_sold
2026-01-01,P001,8
2026-01-01,P002,15
2026-01-02,P001,6
2026-01-02,P002,12
```

These excerpts illustrate the schemas, not complete input datasets.

• Product identifiers must be unique. Costs and selling prices must be positive; stock and sales quantities must be non-negative integers.

• Dates must be valid ISO dates. Each sales record must reference an existing product, and each product-date pair may occur at most once.

• Supply at least 30 consecutive days for every product, with explicit zero-sales rows. Missing product-date pairs are incomplete input, not implicit zero sales.

• Assume historical sales represent demand because the fictional shop had no historical stock shortages. State this simplification in the report.

Validation and persistence

Invalid catalogues or incomplete sales histories must prevent simulation and produce a validation report. Report malformed rows with enough detail to locate and correct them. Repeated imports must not duplicate historical sales. Document treatment of conflicting existing records.

Use related SQLite tables for products, historical sales, simulation runs and daily simulation results. Use primary keys, appropriate constraints, parameterised queries and transactions.

Required SQL summaries

• Units sold and revenue by product.

• Revenue by category using a JOIN.

• Best-selling products in a requested date range.

• A summary of saved simulation runs.

Demand generation with NumPy

Estimate each product's mean daily demand from historical data. Generate a non-negative integer day-by-product array using a Poisson distribution with that mean. No derivation of the distribution is required: explain how it represents varying counts around an average.

Every policy in a comparison must receive the same generated demand array, so performance differences are not caused by different random demand. Use a fixed seed for reproducibility.

Student specifications: simulation and extensions

Daily event order

1. Receive orders placed at the end of the previous day.
2. Observe demand and sell the minimum of demand and available stock.
3. Record unmet demand as permanently lost sales.
4. Charge holding cost on end-of-day stock.
5. Apply the replenishment policy and place orders for the next morning.

Treat each product's replenishment as a separate order when charging fixed order cost. Document how orders placed on the final day are reported.

Measures and inheritance

Report revenue, cost of goods sold, gross profit, holding/ordering costs, operating result (gross profit minus holding/ordering costs), units sold/lost, demand fulfilment rate and final stock, by product and for the shop. Fulfilment rate is units sold divided by demand; zero demand is not applicable. Report final inventory value separately; do not deduct unsold units again as cost of goods sold.

Implement a base replenishment-policy class and two subclasses: **fixed quantity** (below a threshold, order a fixed amount) and **order up to** (below a threshold, order enough to reach a target). The engine must use their common interface without duplicating simulation logic.

Choose exactly one extension

A. Promotion experiment. Apply a percentage discount and assumed demand multiplier during a configured period. Compare revenue, operating result and lost sales with no promotion. The multiplier is an assumption, not an estimated causal effect. Document generation of promotion demand; use identical demand across policies within each condition.

B. Storage constraint. Enforce maximum total stock capacity, with one storage unit per item. Allocate constrained orders using a documented rule and compare at least two allocation rules. Enforce capacity before ordering; reject initial stock exceeding capacity.

C. Policy tuning. Search a finite grid of policy settings. Select on one set of demand simulations and evaluate on fresh simulations. Limit grid size and run count. Explain why using the same simulations for selection and evaluation can be overly favourable. No machine learning or advanced optimisation is required.

Required output files

File	Content
historical_summary.csv	Historical sales and revenue summaries
daily_simulation.csv	Daily stock, demand, sales, orders and financial results by run, policy and product
policy_comparison.csv	Aggregate measures and extension comparisons
validation_report.txt	Input problems and validation outcome
report.txt	Policy comparison, supported recommendation, assumptions and at least two limitations

4. Coding guidelines

These guidelines form part of the student specification.

Architecture and Python

• Separate configuration, validation, database access, domain logic, numerical calculations and reporting. Keep calculations independent of command-line interaction.

• Use focused functions, descriptive names, docstrings for public interfaces and type hints on principal functions and methods. Avoid mutable global state.

• Use fundamental Python types appropriately: dictionaries for keyed records, lists or tuples for collections, and sets where uniqueness matters. Explain these choices.

• Use classes for meaningful state or behaviour and inheritance for genuinely different behaviour behind a common interface. Avoid duplicated simulation logic.

Data, SQL and numerical work

• Document CSV schemas, units, date formats and rounding rules. Choose and justify a consistent monetary representation. Integer cents are suitable for stored amounts; averages may use floating-point values with an explicit rounding policy.

• Use NumPy for substantial array operations or simulations. A conversion to an array and immediately back again is insufficient.

• Use SQL for the required summaries, with primary keys, appropriate constraints, parameterised queries and database transactions. Do not perform all aggregation in Python.

• Use context managers for files and database connections. Catch specific exceptions where recovery or a useful explanation is possible; never silently discard errors.

• Record the effective configuration and random seed with simulation results. Document whether repeated commands replace, append to or create output.

Dependencies and scope

Use the standard library, NumPy and SQLite, plus a testing framework if desired. Python 3.11 or later provides tomllib for TOML input. An ORM must not hide the required SQL. Pandas is unnecessary and should remain outside the core implementation. A graphical interface is unnecessary and earns no additional credit.

5. Testing, teamwork and submission

Automated tests

Test a small example with independently calculated numerical answers; invalid configuration; malformed or inconsistent input; duplicate imports; permitted empty or zero-activity cases; and at least one boundary condition for the selected extension. Test reproducibility when randomness is used. Verify stock conservation, non-negative stock, and the timing of replenishment arrivals.

Tests must check behaviour or numerical correctness, not merely execute code without assertions.

Teamwork and AI use

Each member must make a substantive implementation contribution and review another member's work. Documentation, presentation, database work or testing must not become someone's only contact with the code. All members must understand the complete application.

AI tools are permitted. Submit a brief description of how they were used, including an example of generated output that the group corrected or verified. Complete prompt transcripts are unnecessary. AI use itself neither increases nor decreases the mark. Students must explain and adapt all submitted code during an individual oral examination without AI assistance.

Submission checklist

• Source code, dependency instructions and a working TOML configuration.

• Sample input files, representative outputs and automated tests.

• README with exact commands that reproduce the demonstration.

• Short design note: architecture, inheritance, database structure, modelling assumptions and limitations.

• Contribution statement identifying components implemented and reviewed by each member, plus shared design decisions; AI-use statement.

• Slides for a five-minute presentation.

Five-minute presentation

Time	Content
45 seconds	Problem and selected extension
75 seconds	Architecture and one meaningful design decision
2 minutes	Demonstration and interpretation of results
1 minute	Limitations and lessons learned

6. Operational conventions and assessment

Reproducible execution

Use UTF-8 text files and CSV files with a header row. Document your exact column names, numeric formats and command syntax. Example input rows in this handout illustrate the schema only; your submission must include a complete valid sample dataset.

Your README must explain how to install dependencies, select a TOML file, initialise the database, import data, run the analysis and reproduce the submitted outputs. Include the Python version and dependency versions. Paths and scenario parameters must come from configuration rather than be hard-coded.

Keep distinct simulation runs and scenarios identifiable in the database and outputs. The same inputs, configuration and random seed must reproduce the same numerical results. Never count an unchanged imported record twice.

Simulation conventions

Use the same historical date window for every product. Start day 1 with the configured initial stock and no pending replenishment. Demand, stock and orders are non-negative integers; place no order, and charge no ordering cost, when the requested quantity is zero.

Validate non-negative thresholds and costs, positive fixed order quantities, and an order-up-to target at least as large as its threshold. The trigger is strictly below the threshold. Record final-day orders and their costs if placed; report them as outstanding orders, separate from final on-hand inventory.

Use comparable starting stock, costs and horizons across policies. For storage constraints, account for all ordered units arriving the next morning. For promotion experiments, validate the discount and demand multiplier. For tuning, keep evaluation simulations separate from selection simulations.

Assessment and oral examination

Coding and code quality: 40%. Assessment considers correctness, architecture, meaningful inheritance, file/configuration handling, SQL, NumPy, the extension, exceptions, tests, readability and reproducibility.

Five-minute group presentation: 10%. Explain the problem and assumptions, a meaningful design decision, the results and limitations. Focus on evidence from your application.

Each student must explain both their own work and other parts of the application in an individual oral examination, without AI assistance. Be prepared to trace a record from input to output, explain a query or array calculation, identify a limitation, and make or describe a small change. Follow the course announcements for examination arrangements, deadlines and submission location.